Project Explanation and Viva Study Guide

How the drone traffic-monitoring and digital-twin system works, from the drone camera to the planner's recommendations. Every implementation detail here was checked against the code (2026-10-15). Where something is only planned, or not in the code, it says so.

Part 1 — The story, end to end

1.1 The four "worlds" of the system

Keep these four apart when you explain the project. Examiners often probe exactly this.

World	What lives there	Where in the repo
Physical / simulated world	Roads, cars, traffic lights and a drone with a camera. Today that's **CARLA (the CarlaAir build)**. Real drone videos are processed offline.	`simulation/carla_scripts/`
AI processing pipeline	Pixels in, vehicle tracks, positions in metres, speeds, and violation events out.	`ml/violation_engine/`, `services/live/`
Digital twin	A 3-D model of the same town (lanes, heights, bridges, buildings), with live or replayed vehicles, violation pins, hotspots, zones and lights on top.	`frontend/src/twin/` (CesiumJS)
Recommendation engine	Reads the stored violation history, clusters it into hotspots, and proposes countermeasures using rules.	`ml/planning/`

The core idea: **the whole engine works in metres and seconds, not pixels.** CARLA towns have exact maps (OpenDRIVE), so the AI's coordinate frame, the lane map's frame and the twin's frame are all the same CARLA world frame. That's why mapping a violation onto the twin isn't a guess: the event's `(x, y)` is already a map coordinate.

1.2 The worked story: a car parks in a no-stopping zone

Step 0. A planner draws a zone. In the dashboard twin, a planner draws a polygon in front of a school gate on Town05, type `no_parking`, grace 30 s. It's sent as `POST /api/zones` and stored in PostGIS (`zones` table, Polygon, SRID 0 = map metres) with version history. The backend also writes `backend/data/zones/Town05.json`. The running detector checks that file every 2 s (zone_watch.py) and swaps the zone into the engine **without a restart**.

Step 1. Capture. The drone hovers about 67 m up, camera straight down (pitch −90°), 1920×1080, FOV 90°. stream_flight.py runs inside CARLA. For every rendered frame it sends a JPEG plus a JSON header: frame number, **CARLA sim time**, and the **camera's exact 6-DoF pose** `[x, y, z, pitch, yaw, roll]`. The transport is a single TCP connection with length-prefixed messages (transport.py), with a "latest frame wins" policy: if the GPU is busy, old frames are dropped so delay can't build up.

Step 2. Detection. pipeline.py decodes the JPEG and runs our fine-tuned **YOLO26l** at 960 px with confidence threshold 0.1. It produces axis-aligned boxes for `car / bus / truck`. A class-agnostic NMS pass removes the same car being reported twice under two classes.

Step 3. Tracking. The same `model.track(..., persist=True)` call runs **TrackTrack**, extended with our "buffered IoU" patch. It gives each box a tracker ID that stays the same from frame to frame. The taxi is now "tracker ID 41".

Step 4. Pixels to metres. Using the pose that came with the frame, each box centre becomes a ray from the camera. The ray is intersected with the road surface (taking bridge and road heights from the lane map), which gives world `(x, y)` in metres. The taxi is at `(-55.1, 6.4)`.

Step 5. Clean motion. A **constant-acceleration Kalman filter on ground metres** (online.py) turns noisy positions into smooth position, speed, heading and speed uncertainty. Rows are released **1 s late** (fixed-lag smoothing), so a short backward smoothing pass can run first. If the tracker hands out a new ID where a lost track was predicted to be, the filter keeps the old identity ("relink").

Step 6. Map knowledge. Each smoothed row is matched to a lane by position only (lane id, distance along `s`, sideways offset `d`). It's also tested against every zone polygon. The taxi is "inside zone `school_gate`, lane `r12_l-1`".

Step 7. Rules over time. The `no_parking` monitor asks two questions each frame: is the speed under 2 km/h, and have all positions of the last 3 s stayed within 1 m of each other? If both hold, the **spot's** dwell timer grows. The timer belongs to the place, not the track ID, so an ID switch doesn't reset it. After **30 s** an **event opens**: `Town05live-0007`, type `no_parking`, condition `B3`.

Step 8. Evidence and storage. The event is posted immediately (`POST /api/events`, tagged `provisional`). The evidence recorder saves a snapshot and a clip from 6 s before to 4 s after the flag, with the taxi marked red, and uploads both to S3 (SeaweedFS). When the taxi drives off, the event **closes**. Final confidence and status are computed then (`flagged` or `needs_review`) and sent with `PUT /api/live/events/{id}`.

Step 9. Twin and dashboard. The backend publishes the event on Redis pub/sub, which pushes it over the WebSocket. Within about 1.2 s of the flagged frame (measured median), the Live page shows the taxi's box in red. A pin appears on the 3-D twin at `(-55.1, 6.4)`, and the Violations page lists the event with its clip. An **officer** reviews it and marks it `confirmed` or `dismissed`.

Step 10. Hotspots and recommendations. Over many sessions, events pile up in PostGIS. When a planner opens the Recommendations page, the backend runs recommend.py. **DBSCAN** (radius 50 m, at least 10 events, run per violation type) finds that this gate has 14 no-parking events, which makes it a hotspot. Rule `STOP-ZONE` proposes "Enforce the stopping restriction, or provide a designated stopping bay". Priority comes from a transparent formula, and the projected impact is a sourced published factor. The planner accepts, rejects or modifies it, and the decision goes into `planner_history`.

Each stage's output is the next stage's input:

> JPEG + pose → boxes → boxes with IDs → (x, y) in metres → smoothed (x, y, v, heading, σ) → + lane / zone → rule episodes → events → DB + WebSocket → twin / dashboard → DBSCAN groups → ranked recommendations.

Part 2 — Technical deep dive

A. Capture and transport

- **Purpose:** get frames, the exact camera pose and a common clock to the detector.
- **Input / output:** CARLA camera images → `hello` (width, height, fov, map, ground_z, traffic-light list), then one `frame` message per image, plus `signal` messages when a light changes.
- **Implementation:** a 12-byte binary header (`b"AVF1"`, header length, payload length), then a JSON header and the JPEG. Standard library only, because it must import in both the simulator's Python 3.10 environment and the main venv. The sender reconnects automatically. On the receiving side an IO thread keeps only the newest frame.
- **Why latest-frame-wins:** if a 30 fps camera feeds a 13 fps detector, a queue would grow without limit. Dropping frames keeps delay bounded, and a 2-machine run behaves like a 1-machine one. **Trade-off:** fewer frames reach the tracker, so cars move further between processed frames. That's the reason for buffered IoU in section C.
- **Clock:** every vehicle `t_s` and every light change is CARLA **sim time**. Wall-clock time is used only to measure latency. Real video uses each frame's own timestamp (PTS, via ffprobe), never `frame / fps`, because phone recordings can have a variable frame rate (real_geometry.py).

Status: working. Tested on one machine and on two machines.

B. Object detection

- **Model: YOLO26l** (Ultralytics 8.4.116). It started from COCO weights, was fine-tuned on VisDrone2019-DET, then **retrained on 3 classes** (`car` including van, `bus`, `truck`) using VisDrone, UAVDT and our CARLA frames (`retrain_v1`). Weights: `ml/data/results/retrain_v1/train/weights/best.pt`.
- **Boxes are axis-aligned, not oriented (OBB).** An OBB model (`yolo26l-obb` on DOTA) was evaluated and dropped. The tracking datasets (VisDrone-MOT, UAVDT) only have axis-aligned labels, and DOTA has no video sequences (First_Phase_Plan.md). The exploration is kept in `ml/detection/obb_exploration/` as a documented dead end.

How a box is produced: 1. The frame is resized so the long side is 960 px. A CNN backbone extracts feature maps at several scales: small cars are found on the fine maps, buses on the coarse ones. 2. At each cell of each map, the head predicts box geometry (centre, width, height), one score per class, and an objectness-style confidence. 3. YOLO26 has an **NMS-free, end-to-end head**: during training it learns one prediction per object (one-to-one assignment), so in theory no Non-Maximum Suppression is needed. 4. **Our quirk fix:** in practice the head sometimes gives the same vehicle twice, as `car` and `van`. dedupe_boxes() runs `torchvision.ops.nms` with IoU 0.5 **across classes**, keeping the higher-confidence box. Without this, the tracker would match one box and start a short-lived fake track from the other. 5. **Output per box:** `xywh` (centre x, centre y, width, height in pixels), `conf` (0–1), `cls` (class index). The detector threshold is deliberately low (`DETECT_CONF = 0.1`), because our vehicles typically score 0.1–0.5. Filtering is left to the tracker's two-level thresholds.

Measured: VisDrone val mAP50 0.696 (3-class), GT clip 0.702, CARLA Town05 test 0.844. Inference about 12 ms/image on an RTX 4060 laptop GPU.

Status: working, with a known gap. It's below the PRD's mAP 0.75 target on VisDrone. Recall drops for cars half-hidden under trees.

C. Object tracking

Two separate layers keep vehicle identity, and you should name both.

C.1 Layer 1: TrackTrack, in pixels (the Ultralytics implementation plus our patch)

The config is tracktrack_live.yaml for live runs and `tracktrack_ours.yaml` for full-rate offline runs. The core is Ultralytics' `trackers/track_tracker.py`, subclassed in buffered_tracktrack.py.

Per frame:

1. **Split detections by confidence:** high (≥ 0.15), low (0.05–0.15), and "recovered" boxes (ones a looser NMS kept but the strict NMS dropped).
2. **Camera-motion compensation (GMC, `sparseOptFlow`):** corner features are tracked between frames with Lucas–Kanade optical flow, an affine transform is fitted to them, and every track's predicted box is shifted by that transform. This is how the tracker copes with a moving drone.
3. **Kalman predict, in pixels:** every track has an 8-value state `[x, y, w, h, vx, vy, vw, vh]` (`KalmanFilterXYWH`, constant velocity). Noise is scaled by box size. On update, the measurement noise is scaled by `max(1 − conf, 0.05)` ("NSA-Kalman"), so a confident box pulls the estimate harder.
4. **Cost matrix** (rows = tracks, columns = detections): `cost = (1 − HIoU × IoU) + 0.1 × |projected score − det score| + 0.1 × angle distance`. Any pair with IoU ≤ 0.1 gets cost 1, meaning "impossible". Low-confidence and recovered detections get penalties of +0.2 and +0.4. ReID appearance features are **off** (`with_reid: False`).
5. **Assignment: iterative mutual-nearest greedy matching, not the Hungarian algorithm** (`_iterative_associate`). A pair matches if each is the other's cheapest option and the cost is below a threshold. The threshold starts at 0.7 and drops by 0.05 every round.
6. Unconfirmed new tracks get a second pass against the leftover high-confidence boxes. Lost tracks get a relaxed rebinding pass (`lost_match_thr` 0.9).
7. **Track-aware initialisation (TAI):** a leftover box starts a new track only if its score is above 0.35 **and** it overlaps no existing track by more than IoU 0.45. This stops duplicate boxes and snow blobs from spawning fake IDs.
8. A lost track is kept for `track_buffer` = 90 frames before it's removed.

Our patch, buffered IoU (C-BIoU). Live, only about 8 of every 30 frames are processed, so a car at 100 km/h moves about one car length between them. Its boxes stop overlapping, IoU drops to 0, and the car is never confirmed. `iou_buffer: 1.0` enlarges both boxes by their own width and height on each side before computing IoU. Measured on a Town04 highway recording: cars in view that got a box went from **41% to 78%** (the detector alone reaches 78%), and cars at 70–100 km/h went from 1% to 68%.

Worked example: IoU, HIoU and the buffer. Boxes are `[x1, y1, x2, y2]`. The track's predicted box is A = `[100,100,140,160]` (40×60). - Normal case: detection B = `[110,105,150,165]`. The overlap is 30 × 55 = 1650, so the union is 2400 + 2400 − 1650 = 3150 and **IoU = 0.524**. The vertical overlap is 55 out of a vertical span of 65, so **HIoU = 0.846**. Cost = 1 − 0.524 × 0.846 = **0.557**, under 0.7, so it matches. - Fast car: B = `[100,140,140,200]`, moved 40 px down. Plain IoU is 800 / 4000 = 0.2 and HIoU = 20 / 100 = 0.2, so cost = 1 − 0.04 = **0.96**: no match, and a new ID would be issued. - With buffer 1.0: A′ = `[60,40,180,220]` and B′ = `[60,80,180,260]`. The overlap is 120 × 140 = 16 800 and the union 26 400, so IoU = 0.636, HIoU = 140 / 220 = 0.636, and cost = 1 − 0.405 = **0.595**. It matches.

Worked example: why greedy mutual-nearest isn't Hungarian. Costs: T1→D1 = 0.2, T1→D2 = 0.5, T2→D1 = 0.3, T2→D2 = 0.6. - Round 1 (threshold 0.7): T1's cheapest is D1, and D1's cheapest is T1. They're mutual, so

they match. T2's cheapest is also D1, but D1 prefers T1, so T2 waits. - Round 2 (threshold 0.65): T2→D2 = 0.6 < 0.65, so it matches.

The **Hungarian algorithm** would instead find the assignment with the minimum *total* cost across all pairs, in $O(n^3)$. The greedy method commits the most unambiguous pairs first and gets stricter each round. It's simpler and is what TrackTrack (CVPR 2025) specifies. Its downside is that it can be slightly sub-optimal in crowded scenes. If asked: ByteTrack and BoT-SORT, which we used earlier, *do* use a Hungarian-family linear-assignment solver. TrackTrack, our current tracker, does not.

Why TrackTrack: on the GT clip it scored final IDF1 0.874 against 0.859 for BoT-SORT, with 2 ID switches against 3. After retraining the detector, the GT clip gives **HOTA 0.639, IDF1 0.881, MOTA 0.826, 2 ID switches, 40/40 IDs**. History: ByteTrack → BoT-SORT-ReID → TrackTrack.

C.2 Layer 2: identity and motion in metres (online.py live, kinematics.py offline)

Even a good tracker switches IDs. The second layer fixes this in ground coordinates:

- **Relinking:** a new tracker ID that appears within 3 s of a lost track, with a compatible class, and within `2.5 m + 0.3 × speed × gap` of where the lost track should be by now (constant velocity), **continues the old track**.
- **Splitting:** if one ID jumps impossibly (two consecutive measurements imply more than 150 km/h, or a velocity change above 15 m/s² plus noise slack), it was two cars. The track is cut, and the later piece gets ID `src × 1000 + n`.
- **Confirmation:** a track feeds the rules only once it is at least 1.0 s long and its mean confidence is at least 0.3.
- **Offline**, two extra steps run first: stitch_tracklets.py (greedy linking by motion-compensated position, box size, class and HSV colour histogram) and postprocess_tracks.py (confidence-weighted class vote, dropping short tracks, filling gaps up to 1 s by interpolation).

C.3 The Kalman filter from first principles (the ground filter)

State per axis: `[p, v, a]`, meaning position (m), velocity (m/s) and acceleration (m/s²). x and y are filtered separately. The model is **white-noise jerk**: acceleration drifts randomly, with strength `JERK_Q = 2.0`.

```
Predict (time step dt):   x⁻ = F x          P⁻ = F P Fᵀ + Q
    F = [[1, dt, dt²/2],
         [0,  1,   dt ],
         [0,  0,    1 ]]
Update (measured position z, noise R = 0.15² m²):
    innovation y = z − p⁻        S = P⁻[0,0] + R
    K = P⁻[:,0] / S              x = x⁻ + K·y        P = P⁻ − K·P⁻[0,:]
```

In plain words: **predict** where the car should be from its last velocity, and let the uncertainty `P` grow because time has passed. **Measure** where the detector says it is. **Blend** the two: the Kalman gain `K` is large when the prediction is uncertain compared with the measurement, and small when the prediction is trusted. `P` shrinks after each measurement.

Worked example (simplified to two states `[p, v]` so the numbers are easy to follow; the real filter adds acceleration): - Before: p = 10.0 m, v = 10 m/s, variances 0.04 m² and 1.0 (m/s)². Time step dt = 0.1 s. - **Predict:** $p^- = 10 + 0.1 \times 10$ = **11.0 m**. Position variance grows from 0.04 to about **0.051** (σ ≈ 0.23 m). - **Measure:** z = 11.3 m, R = 0.0225. Innovation y = 0.3, S = 0.073. - **Gain:** K = [0.69, 1.50]. The filter trusts the measurement

about 69% for position, and also corrects velocity, because "further than predicted" means "faster than I thought". - **Update:** p = 11.0 + 0.69 × 0.3 = **11.21 m**, v = 10 + 1.50 × 0.3 = **10.45 m/s**. Position variance falls to **0.016** (σ ≈ 0.12 m). - **Outlier gate:** the measurement is accepted only if $y^2 \le 5^2 \times S = 1.83$. A glitched box reading z = 13 m ($y^2 = 4$) is **rejected**, and the filter predicts through it. After 3 rejections in a row it accepts anyway (the filter must be the thing that's wrong). There is no gate during a track's first 0.5 s, because velocity starts at 0 and the first real measurements of a moving car would all look like outliers.

Why it matters: at about 27 fps, 0.1 m of position noise turns frame-to-frame differences into **±10 km/h** of speed noise. The smoother brings the measured error down to a **median of 1.4 km/h** (p95 5.8 km/h) and heading error to 0.5° median, against CARLA's true speeds.

RTS smoother: after the forward pass, a backward pass corrects each state using later frames: `x_k ← x_k + C_k (x_{k+1} − x⁻_{k+1})`. Offline it runs over the whole track. Live, it runs only over the last 1 s (`LAG_S`). That's why live rows reach the rules 1 s late: a deliberate trade of 1 s latency for cleaner speeds.

Also produced: `speed_sigma_kmh`, the speed's uncertainty, taken from the velocity covariance. Speeding uses it directly. Heading is left blank below 3 km/h, where a near-stopped car's direction is just noise.

Visibility rule: a box touching the frame edge has a wrong centre, so it is **not used as a measurement**. A row counts as `visible = 1` only if a real measurement lies within 0.4 s on both sides of it. The moving-vehicle rules trust only visible rows.

D. Digital-twin mapping (pixels → world → twin)

CARLA path: exact ray casting, no homography (ground_coords.py):

1. Focal length: `f = W / (2·tan(FOV/2)) = 1920 / (2·tan 45°) = 960 px`.
2. Ray direction in the camera's frame (Unreal axes: x forward, y right, z up): `d_cam = (1, (u − W/2)/f, −(v − H/2)/f)`.
3. Rotate into the world with the streamed pose: `d = R · d_cam`.
4. Intersect with the horizontal plane at road height + 0.75 m. A box centre seen from above sits about mid-height of the car (1.85 m for buses and trucks, a value we measured): `t = (z_plane − z_cam) / d_z`, then `(x, y) = cam_xy + t·d_xy`.

Worked example: the camera hovers 67.6 m above the road and looks straight down. A car's box centre is at pixel u = 1440, 480 px right of the image centre. The height difference is 67.6 − 0.75 = 66.85 m, so the ground offset is (480 / 960) × 66.85 = **33.4 m** right of the point directly under the drone. One pixel is about 66.85 / 960 ≈ **7 cm** on the ground.

Road-surface version (`to_ground_on_roads`): lane heights are rasterised into a 0.5 m grid (`RoadSurface`). The ray is tested against height levels every 0.2 m, and the **highest** level where it lands on a road of that height wins, because a flyover deck hides the street below. Measured: on raised roads, median error dropped from **4.91 m to 0.26 m**. On flat roads, projection error against CARLA truth is median 0.11–0.12 m and p95 0.74–0.88 m.

Real footage: homography (scene_map.py, real_geometry.py). With no camera pose, positions take two steps: 1. **Stabilise:** every frame is registered to a keyframe using SIFT features (vehicles masked out) and a RANSAC homography. Keyframes are chained, so every pixel maps into the reference frame. 2. **Calibrate:** reference-frame pixels are converted to metres, using a `scale` (from a known length such as a 3.5 m lane), a `homography` (4 or more ground points with known positions; `cv2.findHomography`), or DJI `.SRT` telemetry (altitude + FOV).

A homography is a 3×3 matrix H acting on homogeneous coordinates: `[x', y', w]ᵀ = H·[u, v, 1]ᵀ`, then `x = x'/w`, `y = y'/w`. Example: H = `[[0.05,0,-10],[0,0.06,-5],[0,0.0001,1]]`. Pixel (400, 300) gives x' = 10, y' = 13, w = 1.03, so the point is **(9.71, 12.62) m**. The bottom row is what lets it model a tilted camera: points further away get compressed. **RANSAC** fits H on random minimal sets of matches and keeps the H that the most matches agree with, so moving cars and bad matches can't skew it. Real-footage speeds are tagged `estimated_speed`.

Lane-map matching (lane_map.py): - The lane map is JSON exported automatically from CARLA's OpenDRIVE (export_lane_map.py). For each lane it holds the centreline points (whose order gives the direction), width, type, speed limit, line types, junction flag, successor lanes, heights, and bridge / ramp / highway flags. - A KD-tree finds nearby centreline segments. The point is projected onto each segment, giving `s` (distance along the lane) and signed `d` (sideways offset; the sign is flipped because CARLA's axes are left-handed). The lane with the smallest |d| that the point lies inside, plus 0.75 m slack, wins. - **Matching uses position only, never heading.** A wrong-way car must be matched to the lane it's physically in. If heading were used, it would match the opposite lane, agree with it, and never be flagged. - For speeding only, `drive_lane` uses the nearest lane running the car's way. This stops a car on the centre line from getting the oncoming lane's limit. - Zones are Shapely polygons; `zones_at(x, y)` is a point-in-polygon test.

Into the twin (frontend/src/twin/coords.ts): CARLA (x, y, z) becomes local East-North-Up `(x, -y, z)` (the y flip converts a left-handed frame to a right-handed one). That becomes Earth-centred coordinates through `Transforms.eastNorthUpToFixedFrame(anchor)`. The default anchor is **lat 0 / lon 0**, the georeference CARLA writes into its towns. **There is no real GPS:** a real site would need a surveyed anchor passed with `?anchor=lat,lon`.

What the twin shows: lanes in 3-D with heights and markings, junctions, crosswalks, the town's buildings and poles, vehicles (live over the WebSocket, or replayed from stored trajectories) coloured ok / checking / flagged, 3-second trails, violation pins, hotspot columns, a list of problem sections, zones (which can be drawn and edited), traffic lights, lane edits saved as profile versions, and what-if scenarios.

E. Violation detection

Design: reusable **predicates** plus a **time condition**, the "temporal logic" pattern from Maierhofer et al., IEEE IV. See predicates.py and rules.py. PRD thresholds given in frames are stored in seconds (5 frames at 30 fps = 0.17 s), so they hold at any frame rate.

The `Episode` object: how temporary detections are filtered out. - It opens when the condition first holds. - It becomes an event only after holding for `min_s`. - While still pending, gaps up to `confirm_gap_s` are tolerated. Once an event is open, gaps up to `close_gap_s` are tolerated. - One episode is one event, so the same incident can't be recorded twice. The event closes when the condition has been false for longer than the gap.

Key predicates: - `stopped`: **stay-point test.** All positions in the last 3 s lie within 1 m of their mean. This is more robust than "speed < X", because box jitter makes a parked car's speed read a few km/h. - `queue_context`: a slow vehicle within 10 m directly ahead in the same lane (or the next piece of it). It explains a stop as traffic rather than a violation. - `against_lane_angle`: angle between heading and lane direction, 0–180°. - `straddle_overlap`: `|d| + half vehicle width − lane width/2`. - `speed_tolerance`: 5 km/h below 100 km/h, 5% above (EU practice).

Type (condition id)	Exact rule in the code
No-parking (B3)	Inside a `no_parking` zone, under 2 km/h **and** stay-point, for **≥ 30 s** (a zone can override this with `grace_s`).
Zebra crossing (F1)	Inside a `crosswalk` zone, stopped, **≥ 10 s**. If at least 50% of its frames were in a queue, it is tagged `queue` and **suppressed**.
Highway stop (B1 / B2 / B4 / B5)	On a highway-class, ramp, bridge or tunnel lane (the road itself acts as the zone), under 5 km/h, **≥ 20 s**. Queue → suppressed. Mostly on the shoulder → `possible_breakdown`.
Wrong-way (C1–C5)	Visible, on a non-junction driving lane, angle **> 150°**, speed > 5 km/h, held **≥ 0.17 s and ≥ 5 m** driven backwards along the lane. Tags for entry, divided road, ramp, one-way.
Speeding (E1 / E3 / E4)	`speed − 2σ > limit + tolerance` for **≥ 0.33 s**. Track at least 1 s old, visible rows only. The limit comes from a speed zone, otherwise the lane, capped by any class limit.
U-turn in a zone (D1)	Small heading changes summed inside a `no_u_turn` zone reach ≥ 160°. A jump > 90° between consecutive headings means the car reversed: a **three-point turn**, which is suppressed.
U-turn anywhere (D2–D4)	Turns ≥ 160° within 30 s and settles on the opposite lane of the **same road**. Illegal only across a solid centre line, through a median, or at a listed junction.
Lane violation (A1–A8)	Straddling a marked line by ≥ 0.3 m for > 3 s (A4); a completed lane change over a solid line (A2); a lane change forbidden by `lane_change` (A3); a lane after a junction that can't be reached from the lane before it in the lane graph (A1); a restricted lane (A5); driving on the shoulder (A6); TTC < 2 s to the lead or lag car for ≥ 0.3 s after a lane change (A8, `high_risk` if < 1 s).
Red light (V4)	The front bumper crosses a stop line, in the approach direction, on red, then gets 10 m past it before stopping. **Off by default**, CARLA only.

Worked example of the temporal logic (wrong-way): a car at 28 km/h (7.8 m/s) goes against the lane at 176°. After 0.17 s it has covered only 1.3 m, so the time condition is met but the distance condition isn't. At about 0.65 s it passes 5 m backwards, and the **event opens** with `flag_s` set to that moment. A car reversing 2 m into a parking space never reaches 5 m, so it is never flagged. That's the PRD edge case, enforced by the distance condition.

Repeated detections of the same incident: - Moving rules key episodes by track ID. - Stop rules key them by **spot** (1.5 m radius, kept 30 s). A parked car that gets two IDs, or whose detection drops out under a tree crown for 24 s, still has **one** timer and **one** event, tagged `id_switch_bridged`. The 30 s keep time came from a measured failure: under tree crowns, detections dropped out for 10 s and 24 s.

Confidence and status (events.py): `confidence = 0.4·margin + 0.3·quality + 0.3·duration_score`, each clipped to 0–1. - *margin* is how far past the threshold. For speeding: `(max lower bound − threshold) / max(3σ, 1.5)`. - *quality* is mean detector confidence × the share of frames with a full box. - *duration* is time held divided by twice the minimum time.

Below the PRD minimum for the type (0.90 no-parking, 0.85 speeding, …) the status becomes `needs_review`. Suppressed edge cases are **kept for audit**, not deleted. - **Example:** speeding at 58 ± 1 km/h in a 50 zone. Margin = (56 − 55) / 3 = 0.33, quality 0.6, duration 1.0, so confidence = 0.13 + 0.18 + 0.30 = **0.61**, which means `needs_review`. - **Be honest about this:** the weights are design choices, and the code says "to be calibrated on CARLA". Because our detector scores are low (0.1–0.5), many true events land in `needs_review`. Since every event goes to a human anyway (the same model Spain's DGT drone enforcement uses), that's acceptable, but it's a known calibration gap.

Offline-only extra: static_vehicles.py compares the image patch at the event spot with "empty" views from before and after the event. A confirmed static vehicle adds +0.1 confidence; a contradiction sends the event to review. It isn't in the live path.

Measured: - 0 false events on 3.5 min of normal Town05 traffic (down from 784 before the fixes). - On a staged flight, wrong-way, U-turn and zebra scored F1 1.0. Later staged sessions: 38 of 42 staged acts caught, 0 of 25 negative acts triggered. - Speeding went from F1 0.4 to 1.0 after the lane-matching fixes. - Lane-map matching 99.1%.

The full per-type F1 across all spots is still incomplete. Don't claim "≥ 0.70 for every type".

F. Recording violations and evidence

- **Event shape** follows schemas/event.schema.json: `event_id` (`<session>-0007`), `type`, `condition`, `status`, `confidence`, `track_ids[]`, `cls`, `start_s / flag_s / end_s` (sim time), frames, `lane_id`, `zone_id`, `x / y` (map metres at the flag), `value` (rule-specific, e.g. `dwell_s`, `max_speed_kmh`), `tags`, `evidence`.
- **Live lifecycle:** at the flag, `POST /api/events` with the `provisional` tag and confidence 0. At close, `PUT /api/live/events/{id}` with the final values. Evidence is uploaded to `POST /api/live/events/{id}/evidence?kind=snapshot|clip`, retried until the event exists.
- **Failure handling** (backend_client.py): if the backend is down, event posts are buffered on disk and retried with backoff 1, 2, 4, 8, 16, 30 s. Vehicle states only keep the newest batch, because a stale position is useless. A watchdog restarts a hung pipeline (watchdog.py).
- **Offline path:** `run_violations.py` writes `violations.json / .csv`, and `render_violations.py --clips` makes 10 s clips. The session is imported with `POST /api/sessions/import`.

G. Hotspot detection

There are **two hotspot methods**. Know which one feeds which screen; this is a likely trick question.

	Grid hotspots	**DBSCAN hotspots**
Where	`GET /api/stats` (events.py:23), the Stats page, and the twin's hotspot columns (layers.ts)	Recommendations (analytics.py)
Method	`cell = (floor(x/25), floor(y/25))`, count events per cell, keep the top 10, centre = mean position	DBSCAN, eps **50 m**, minPts **10**, **separately per violation type**; ≥ 50 events = "major hotspot"
Why	Fast and cheap, good for a live display	Clusters of any shape, separates isolated noise, follows PRD 21.1

DBSCAN from first principles (hand-written in analytics.py:84, not sklearn): - **ε (eps):** the neighbourhood radius. **minPts:** how many points, counting itself, make a point "dense". - A **core point** has at least minPts neighbours within ε. - A **border point** is not core, but lies within ε of a core point. It joins that cluster but doesn't grow it. - A **noise point** is neither. It's an isolated event, not part of a hotspot. - **Expansion:** start at an unvisited core point and give it a new cluster label. Push it on a stack. Pop each point; if it's core, label all its unlabelled neighbours and push the ones that are themselves core. When the stack empties, the cluster is finished.

Toy example (ε = 50 m, minPts = 3, to keep it small):

```
A(0,0)  B(30,0)  C(60,0)  D(30,40)  G(100,0)  E(200,0)  F(500,500)
A: {A,B,D}   (D is exactly 50 m away; ≤ ε counts)  → 3 → core
B: {A,B,C,D} → core      C: {B,C,D,G} → core      D: {A,B,C,D} → core
G: {C,G}     → 2 → not core, but within 50 m of core C → BORDER
E: {E}, F: {F} → NOISE
```

Run: start at A (cluster 0) → B and D join → popping D adds C → popping C adds G (border, not pushed). **Cluster 0 = {A,B,C,D,G}; noise = {E,F}.**

Choices you should be ready to defend: - **What gets clustered:** each event's `(x, y)` at its flag time, in map metres, Euclidean distance (via a KD-tree). Only counted statuses (`flagged`, `needs_review`); suppressed queue and breakdown cases are left out. - **Time and severity do not affect the clustering.** Clustering is purely spatial and per type. Time appears afterwards in each group's `period` and in the limitations ("all evidence from 210 s: no trend"). Severity enters later, in priority. - **Recurring hotspot vs unrelated incidents:** at least 10 same-type events within 50 m chains is a hotspot. Leftover noise events are grouped per road (or per zone for zebra and no-parking) with a smaller DBSCAN (minPts 2) as **"emerging"** patterns, explicitly labelled "not a confirmed hotspot". A group from a single session is labelled "recurrence not shown". - **Ranking:** by size; recommendations are then ranked by priority (next section). - **Determinism:** events are sorted by `event_id` before clustering, so the labels don't depend on input order.

Congestion: `section_stats()` computes, per lane and per road, density (vehicles/km over the observed length), mean and 85th-percentile speed, share of samples over the limit, and queue length (stopped vehicles chained if within 12 m of each other). This feeds a `FLOW-QUEUE` recommendation, **but only from the command line** (`recommend.py --kinematics`). The API endpoint calls `recommend(events, scene)` without kinematics, so **the dashboard currently shows no congestion recommendations.**

H. Recommendation engine: rule-based, not machine learning

recommend.py:

1. **Collect:** counted events (for one session, or all of them) and the scene's lane map.
2. **Group:** DBSCAN hotspots per type, plus emerging groups, plus flow groups if kinematics are given.
3. **Context per group:** lane features (bridge, ramp, junction, median, highway), speed limits, crosswalks within 30 m, the dominant condition id, and this type's share of all events nearby.
4. **Rule selection:** an ordered `RULES` list, where the **first rule that matches wins**. For example, speeding: `SPD-HWY` if on a highway, else `SPD-VUL` if near a crossing, else `SPD-URB`. Others: `WW-RAMP / WW-DIV / WW-ONEWAY` (one-way conversion if wrong-way is more than 40% of local events), `UT-MEDIAN / UT-JUNCTION / UT-ZONE`, `STOP-BRIDGE / STOP-RAMP / STOP-HWY / STOP-ZONE`, `ZEB-BLOCK / ZEB-YIELD`, `LANE-UNSAFE / LANE-SOLID / LANE-WRONG / LANE-MARKING`, `RL-SIGNAL`, `FLOW-QUEUE`. Each rule gives the problem text, an action, a countermeasure key, alternatives with trade-offs, and a simulation change.
5. **Priority:** `score = 100 × severity × frequency × evidence quality × tier × objective` - severity: design weights, e.g. wrong-way 1.0, speeding 0.8, no-parking 0.4, with +0.1 on a bridge or near a crossing - frequency: `min(1, 0.3 + 0.7·ln(1+n)/ln(51))` - evidence quality: mean event confidence - tier: major 1.0, hotspot 0.9, emerging 0.5

Bands: high ≥ 50, medium ≥ 25. The explanation string spells out the multiplication.

Example: 14 speeding events near a crossing. Severity 0.9; frequency 0.3 + 0.7 × ln 15 / ln 51 = **0.78**; quality 0.62; tier 0.9. Score = 100 × 0.9 × 0.78 × 0.62 × 0.9 = **39.3, medium**. The rule chosen is `SPD-VUL` (traffic calming plus a 10 km/h limit review). 6. **Expected impact:** a **published** crash-modification factor from

catalogue.py. For a speed camera that's 17–42% fewer crashes, so 14 events project to 8.1–11.6. It's always labelled a "projected estimate", and where no published factor exists, the output says "not quantified". 7. **Confidence:** `mean confidence × tier × (1 − 0.3 × share needing review) × (0.85 if only one session)`. 8. **Validation plan:** baseline vs modified CARLA runs (Traffic Manager speed clamps, no-lane-change, parked-car removal, signal timing). The runner is written (validate_recommendation.py); **the full runs are still pending.** 9. **Learning from decisions:** `history.rerank` multiplies priority by measured outcomes of earlier decisions, shrunk towards "no change" when there are few results. **This isn't wired into the API call.**

Implemented vs proposed: - **Implemented:** the rule table, DBSCAN grouping, priority, sourced projections, accept / reject / modify decisions logged in `planner_history`, and what-if (whatif.py: re-runs the engine on recorded traffic with a changed limit or zone; drivers don't react) plus an IDM micro-simulator (microsim.py, clearly labelled "not CARLA"). - **Not implemented:** any ML-based recommendation, measured CARLA validation results, congestion recommendations in the UI, and history re-ranking through the API.

I. Backend, database and frontend

- **Stack:** FastAPI (async SQLAlchemy plus GeoAlchemy2), **PostGIS 16** (with spatial indexes), **Redis** (live state with a 2 s time-to-live, plus pub/sub), **SeaweedFS** (S3-compatible, in place of MinIO for licensing reasons). All three run in docker-compose.yml.
- **Main tables** (models.py):
- `sessions`: `source` carla|video|live, `town`, `started_at`, `t0_s`. `t0_s` is the sim time at `started_at`, which is how sim time becomes wall-clock `occurred_at`.
- `events`: copied columns for filtering, plus `geom` Point SRID 0, plus the full event in `data` JSONB.
- `reviews`, `tracks` (≤ 5 Hz points), `zones` + `zone_versions`, `profiles` (versioned rule thresholds), `recommendations`, `planner_history`, `scenarios`, `plans`, `reports`.
- `audit_log`: append-only, enforced by a database trigger.
- **Key endpoints** (API.md):
- live pipeline (service token): `POST /api/live/state`, `/frame`, `/signals`, `/events/{id}/evidence`, and `POST /api/events`
- reads: `GET /api/events` (filters including a bounding box), `/api/stats`, `/api/recommendations`
- writes: `POST /api/events/{id}/review` (OFFICER), `POST/PUT/DELETE /api/zones` (PLANNER / ADMIN)
- `GET /api/export?format=pdf|xlsx|geojson|csv`, and `WS /api/ws/live` (vehicles ≤ 10 Hz, events never dropped, 60 s catch-up after a reconnect)
- **Auth:** JWT access token (15 min) plus a single-use refresh token. Five roles: OFFICER, OPERATOR, PLANNER, MAINTENANCE, ADMIN.
- **Dashboard** (`frontend/`, React 19 + Mantine + TanStack Query): Live (raw and annotated MJPEG side by side, event feed), Violations (filters, drawer with snapshot and clip, review buttons), Sessions, Config (profiles), Recommendations, Twin (CesiumJS), and Stats (behind a flag).

Measured live performance: 13.1 fps processed, detect + track 52 ms median, **event latency from flagged frame to dashboard: 1.19 s median, 1.71 s max** (target ≤ 3 s). Vehicle states arrive at 9.5 Hz.

Part 3 — Presentation material

3.1 Two-minute version

> "We built a drone-based traffic-enforcement and planning system, tested in the CARLA simulator. A drone camera streams frames together with its exact pose and the simulator clock. Each frame goes through a fine-tuned YOLO26 detector for cars, buses and trucks. TrackTrack gives each vehicle a consistent ID, with a buffered-IoU change we made so fast highway cars stay tracked at the live frame rate.
>
> Using the camera pose, we cast a ray through each box onto the actual road surface, so every vehicle gets a position in metres in the same coordinate frame as the town's lane map. A Kalman filter turns those positions into clean speed and heading, accurate to about 1.4 km/h.
>
> A rule engine then checks seven violation types — no-parking, wrong-way, U-turns, speeding, lane violations, zebra crossings and highway stopping — as conditions that must hold over time. For example, stopped inside a zone for 30 seconds. Each incident becomes one event with a confidence score and a video clip, and a human officer confirms it.
>
> Events are stored in PostGIS and appear on the dashboard and a 3-D CesiumJS digital twin about 1.2 seconds after the frame. Finally, DBSCAN clusters the violation history into hotspots, and a transparent rule table proposes countermeasures, ranked by severity, frequency and evidence quality, with sourced impact estimates for a planner to accept or reject."

3.2 Five-minute technical version (talking points)

1. **Problem framing:** pixels are meaningless for rules when the camera moves. Everything is converted to metres and seconds. That's the one design decision everything else follows from.
2. **Perception:** YOLO26l, retrained on 3 classes, at 960 px with a 0.1 threshold, plus class-agnostic NMS for the head's duplicate boxes. Axis-aligned boxes, because no tracking dataset has oriented labels.
3. **Tracking:** TrackTrack's flow — GMC, Kalman predict in pixels, the HMIoU + confidence + angle cost, greedy mutual-nearest assignment with a falling threshold, and track-aware initialisation. Buffered IoU for live: 41% → 78% of cars boxed. IDF1 0.881 on our GT clip.
4. **Geometry:** CARLA ray casting onto the road-surface grid (0.11 m median error). Real footage uses SIFT + RANSAC homography stabilisation and calibration, and its speeds are flagged as estimates.
5. **State estimation:** constant-acceleration Kalman filter, a 5σ outlier gate, splitting at impossible jumps, relinking lost tracks in metres, and a 1 s fixed-lag RTS smoother. Show the predict / update example.
6. **Rules:** predicates plus episodes. Walk through no-parking (stay-point, spot-based timer) and wrong-way (angle + time + 5 m). Mention suppression with an audit trail (queue, three-point turn).
7. **Events and evidence:** the confidence formula, the `needs_review` path, the provisional-then-final live lifecycle, and the 10 s WebM clip.
8. **Integration:** FastAPI, PostGIS, Redis pub/sub, WebSocket. The disk-spooled retry buffer and the watchdog. 1.19 s median latency.
9. **Twin:** CARLA frame → ENU (y flipped) → Earth-centred coordinates via a lat 0 / lon 0 anchor. Zones drawn in the twin reach the running engine within 2 s.
10. **Planning:** DBSCAN per type (ε 50 m, minPts 10), emerging patterns, the first-match rule table, the priority formula, published factors, the decision log. Be explicit that this is rule-based.

3.3 Data-flow diagram



3.4 Component table

Component	Technology / algorithm	Input	Output	Purpose	Status
Capture	CARLA camera, TCP length-prefixed JSON + JPEG	Simulator	Frames + pose + sim time	Live feed with geometry	Working
Detection	YOLO26l, retrained 3 classes, class-agnostic NMS	Frame	Boxes, class, confidence	Find vehicles	Working; mAP below PRD target
Tracking	TrackTrack: XYWH Kalman, GMC, HMIoU cost, greedy mutual-nearest, TAI, buffered IoU	Boxes	Tracker IDs	Identity across frames	Working
Geometry	Ray–plane / road-surface intersection; homography (real)	Box centre + pose	(x, y) m	Metric positions	CARLA exact; real = estimate
Kinematics	CA-Kalman + RTS (fixed-lag live), gating, split, relink	Positions	x, y, v, heading, σ	Clean motion	Working (1.4 km/h median)

Component	Technology / algorithm	Input	Output	Purpose	Status
Scene	OpenDRIVE lane-map export, KD-tree, Frenet s/d, Shapely	Positions	Lane, zones	Road knowledge	Working (99.1% lane match)
Rules	Predicates + Episode time conditions	Observations	Events	Detect violations	7 types working; red light off
Events / evidence	Confidence formula, WebM clip	Episodes	Event JSON + media	Reviewable record	Working; weights not calibrated
Backend	FastAPI, PostGIS, Redis, S3, JWT	Events / states	API + WebSocket	Store and serve	Working (31 API tests)
Twin	CesiumJS, CARLA → ENU → Earth-centred	Scene, events, states	3-D view	Situational picture	Working; no real GPS
Hotspots	25 m grid (stats / twin); DBSCAN (planning)	Event (x, y)	Clusters	Find recurrence	Working
Recommendations	First-match rule table + priority formula + published factors	Hotspot groups	Ranked proposals	Planning support	Working; CARLA validation pending

3.5 Likely viva questions with grounded answers

Q1. Why not oriented bounding boxes, when aerial cars are rotated? OBB was tested (DOTA) and dropped because the tracking datasets only have axis-aligned labels, and two detectors would have doubled the training and validation work. From straight above, the box centre is still accurate; only the footprint is looser. The lane-straddle test uses the class's half-width instead of the box.

Q2. Do you use the Hungarian algorithm? No. TrackTrack uses iterative greedy mutual-nearest matching with a threshold that drops 0.05 per round. Hungarian minimises total cost globally; greedy commits the clearest pairs first. Our earlier trackers (ByteTrack, BoT-SORT) did use linear assignment.

Q3. How many Kalman filters are there, and why two? Two. One in pixels inside the tracker (8-value constant velocity) to predict where a box will be for matching. One in ground metres (constant acceleration per axis, with RTS smoothing) for speed and heading the rules can trust. The first serves association; the second serves measurement.

Q4. How do you handle a moving drone? In the tracker, GMC (optical-flow affine) shifts the predicted boxes. For positions, each frame's own camera pose is used in the projection, so a parked car stays at fixed world coordinates however the drone moves. For real footage, frames are registered to a keyframe with a homography.

Q5. What stops a single bad frame from creating a violation? Four layers: the 5σ Kalman gate, the `visible` support rule, minimum durations (0.17 s to 30 s) with the extra distance condition for wrong-way, and the track having to be 1 s old with mean confidence ≥ 0.3 before it's confirmed.

Q6. What if a parked car's ID switches halfway through? The dwell timer belongs to the spot (1.5 m radius, 30 s memory), not the ID. Both IDs feed one timer, and the event gets the tag `id_switch_bridged`.

Q7. Why doesn't a traffic jam on a crossing get flagged? `queue_context`: if at least half of the episode had a slow vehicle within 10 m ahead in the lane, the event is tagged `queue` and suppressed, but kept for audit.

Q8. Why match lanes by position and not heading? A wrong-way driver matched by heading would be assigned to the opposite lane, which it agrees with, and would never be flagged.

Q9. How accurate are positions and speeds? CARLA projection: 0.11–0.12 m median, p95 under 0.9 m. Raised roads 0.26 m. Speed: 1.4 km/h median, 5.8 km/h p95. Heading: 0.5° median.

Q10. Is speeding fair given measurement error? Yes. It requires `speed − 2σ > limit + tolerance` for 0.33 s, so the lower bound of the speed estimate has to be over the limit.

Q11. How is the confidence score computed, and is it calibrated? `0.4·margin + 0.3·quality + 0.3·duration`. It isn't statistically calibrated: the weights are design choices, and the code says they're to be calibrated on CARLA. Events below the PRD minimum go to human review.

Q12. How does a violation get onto the twin? Its `(x, y)` is already in CARLA world metres. The twin converts that to East-North-Up by flipping y, then to Earth-centred coordinates through a lat 0 / lon 0 anchor. No homography is needed in CARLA.

Q13. Do you use GPS? Not for real. The twin's anchor is CARLA's own lat 0 / lon 0. Real georeferencing (orthophoto registration, DJI telemetry) is designed, and the SRT scale option exists, but geo-mapping to GPS is only partial.

Q14. Why two hotspot methods? The 25 m grid is cheap, live, and drives Stats and the twin. DBSCAN finds clusters of any shape, separates noise, follows PRD 21.1, and drives recommendations.

Q15. Why DBSCAN rather than k-means? No need to choose k in advance. It separates isolated incidents as noise. Clusters can follow a road's shape. ε has a physical meaning (50 m). *Weakness:* a single ε for both dense junctions and sparse highways. HDBSCAN would adapt to density.

Q16. Is the recommender AI? No. It's an explainable rule table, applied to statistically grouped data. That was deliberate: a planner must be able to see why something was proposed. The impact numbers are published factors, labelled as projections.

Q17. How does a zone drawn in the UI take effect? `POST /api/zones` → PostGIS → the backend writes the zone file → the pipeline's ZoneWatcher reloads it within 2 s → `SceneMap.replace_zones`, and the next frame uses it.

Q18. What's the end-to-end latency, and where does it come from? 1.19 s median, frame to dashboard. Most of it is the deliberate 1 s fixed-lag smoothing window; the rest is detection (~52 ms) and transport.

Q19. What happens if the backend crashes mid-run? Event posts are buffered (on disk too), retried with backoff 1–30 s, and replayed in order. Live states are just dropped, because stale positions are useless. A watchdog restarts a hung pipeline.

Q20. How did you validate the rules? Three levels: - L0: unit tests on synthetic tracks. - L1: rules on CARLA's true positions (oracle). - L2: the full pipeline on staged CARLA scenarios with logged ground truth (positive and negative acts). Examples: 0 false events in 3.5 min of normal traffic; 38 of 42 staged acts caught with 0 of 25 negatives triggered.

Q21 (hard). Why use the 2σ speed bound but a fixed 5σ gate? They do different jobs. The gate only throws out glitches, so it's loose. The 2σ bound is a decision threshold where a false accusation is costly, so it's conservative.

Q22 (hard). Your detector scores are 0.1–0.5. Doesn't that break the confidence formula? It pushes quality down, so more events go to `needs_review`. That's safe, because a human reviews everything, but it's a calibration issue we'd fix by calibrating detector scores (temperature scaling) or learning the weights from labelled CARLA events.

3.6 Limitations, edge cases and failure scenarios

- **Simulation-first:** results are on CARLA. Real footage works (5 real videos run and checked by eye) but has no true pose, needs manual calibration, and its speeds are estimates.
- **Detection:** in-scope mAP below the PRD's 0.75. Cars under tree crowns or overpasses lose detections, and heavy occlusion is the main cause of misses. No snow in the training data (snow blobs become boxes; TAI and confidence gates reduce this).
- **Tracking at the live frame rate:** latest-frame-wins means large jumps between processed frames. Buffered IoU helps, but in dense traffic it can merge neighbouring cars (the split test catches impossible jumps).
- **Flat-ground assumption** off the mapped roads, and the box-centre height is per class (0.75 m / 1.85 m), not per vehicle.
- **Frame edge:** boxes cut by the edge are excluded, so a violation right at the edge can be missed. This happened with a staged no-parking spot.
- **Lane violations** are the hardest: 0.1–0.8 m position error is a real fraction of a 3.5 m lane. The 0.3 m overlap and 3 s rules reduce false alarms but cost recall.
- **Red light:** off by default and CARLA-only. From above, the lights face the drivers, not the drone.
- **Confidence not calibrated; severity weights are design weights.**
- **Recommendations:** no measured CARLA validation results yet; congestion and history re-ranking aren't exposed in the UI; what-if replay doesn't model drivers reacting; the IDM micro-simulator has no lane changes or signals.
- **Out of scope or dropped:** helmet and overloading detection, the road-surface anomaly objective (code left unused), ReID appearance features, automatic marking detection, real GPS georeferencing.

3.7 Why this architecture is appropriate, and what to improve

Strengths: - One metric coordinate frame shared by perception, rules, map and twin removes a whole class of mapping errors. - Predicate + episode rules are transparent, unit-testable, and configurable per zone and profile, which suits enforcement, where every decision must be explainable. - Two-stage identity (pixel tracker plus metric relink and split) is robust where each alone failed. - The same engine runs offline and live (`Engine.step` is shared), so offline validation carries over to the live system. - Human-in-the-loop review and auditable suppression match how real drone enforcement works.

Where it falls short / next steps: 1. Calibrate event confidence on labelled CARLA events (logistic regression on margin, quality and duration) instead of fixed weights. 2. Improve detection with tiled inference (SAHI), higher resolution, and hard negatives such as snow. A retrain is planned as the final step. 3. Add vehicle ReID or OBB for dense traffic. 4. Real georeferencing: orthophoto registration (Geo-trax style) and a surveyed anchor for the twin. 5. Pass kinematics and history into `/api/recommendations` so congestion and outcome-based re-ranking reach planners. 6. Run the CARLA baseline-vs-modified validations to replace projected impacts with measured ones. 7. Use HDBSCAN or density-adaptive ε, and add time-weighting so recent hotspots rank higher.